

App Dev Project Report

1. Student Details

Name: KUSHALKUMAR BIJAYKUMAR SHAW

Roll Number: 23f2001882

Email: 23f2001882@ds.study.iitm.ac.in

Repo Link: <https://github.com/23f2001882/Placement-PortalApplication-V2>

About Me:

I am a student in the IIT Madras BS Degree Program with a strong interest in full-stack web development and scalable application systems. I enjoy building structured applications that combine backend APIs, modern frontend frameworks, and data-driven interfaces. My focus is on designing modular and maintainable web systems using clean architecture principles.

2. Project Details

Project Title:

Placement Portal Application

Problem Statement:

Campus recruitment processes in many institutes are still managed manually using spreadsheets, emails, and offline coordination. This creates difficulties in tracking company approvals, managing placement drives, verifying student eligibility, and monitoring application status.

The objective of this project is to build a **web-based Placement Portal** that automates the campus recruitment workflow. The system allows institutes to manage companies and placement drives while enabling students to apply for jobs and track their applications.

Approach:

The system was implemented using a **Flask backend with REST APIs and a Vue.js frontend**. The backend handles authentication, business logic, and database operations, while the frontend provides a responsive user interface for different user roles.

The platform supports three main users:

- **Admin** – manages companies and approves placement drives
- **Company** – creates recruitment drives and reviews applicants
- **Student** – browses drives and submits applications

This architecture separates frontend and backend responsibilities and enables scalable application design.

3. AI / LLM Declaration

ChatGPT (GPT-5) was used to assist with:

- Initial **database model structure suggestions**
- Some **frontend component scaffolding**
- Minor improvements in variable naming and documentation formatting

AI tools were used only as development assistance. All core implementation, debugging, integration, and testing were completed manually.

Estimated AI Usage: 10–15%

4. Technologies and Frameworks Used

Technology	Purpose
Flask	Backend REST API framework
SQLAlchemy	ORM for database modeling
SQLite	Database for storing application data
Vue.js	Frontend JavaScript framework
Vue Router	Client-side routing
Pinia	State management
Axios	Communication between frontend and backend APIs
Bootstrap 5	Responsive UI design
Chart.js	Dashboard analytics visualization
JWT	Token-based authentication
Redis	Caching frequently accessed data
Celery	Background task processing

5. Database Schema / ER Diagram

The system contains five main entities.

Tables

User

Stores login credentials and user roles.

Fields: id, email, password_hash, role, created_at

Student

Stores student profile information.

Fields: id, user_id, name, branch, cgpa, graduation_year

Company

Stores company details and approval status.

Fields: id, user_id, company_name, hr_contact, website, approval_status

PlacementDrive

Stores job opportunities created by companies.

Fields: id, company_id, job_title, eligibility_branch, eligibility_cgpa, deadline, status

Application

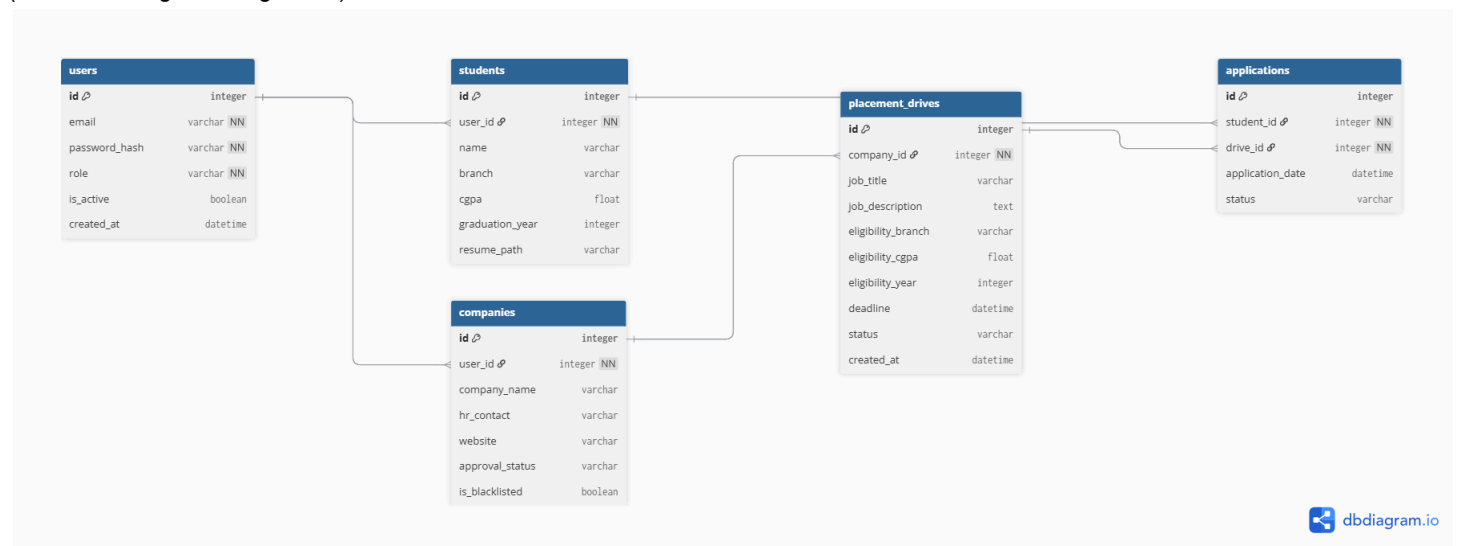
Stores student applications to drives.

Fields: id, student_id, drive_id, application_date, status

Relationships

- User → Student (One-to-One)
- User → Company (One-to-One)
- Company → PlacementDrive (One-to-Many)
- Student → Application (One-to-Many)
- PlacementDrive → Application (One-to-Many)

(Insert ER Diagram image here)



6. API Resource Endpoints

Authentication

Endpoint	Method	Description
/auth/login	POST	Authenticate user
/auth/register/student	POST	Register student
/auth/register/company	POST	Register company

Admin APIs

Endpoint	Method	Description
/admin/dashboard	GET	Fetch system statistics

/admin/companies	GET	List registered companies
/admin/approve-company/{id}	POST	Approve company
/admin/drives	GET	View placement drives
/admin/approve-drive/{id}	POST	Approve drive

Company APIs

Endpoint	Method	Description
/company/create-drive	POST	Create placement drive
/company/my-drives	GET	View company drives
/company/update-application-status	POST	Update candidate status

Student APIs

Endpoint	Method	Description
/student/drives	GET	View available drives
/student/apply-drive/{id}	POST	Apply for job
/student/my-applications	GET	Track applications

7. Architecture and Features

System Architecture

The system follows a **separated frontend-backend architecture**.

Backend (Flask)

Handles authentication, business logic, and database interaction.

Main modules:

- models – database entities
- routes – API endpoints
- services – business logic
- cache – Redis caching
- jobs – background tasks

Frontend (Vue.js)

Provides user interface and communicates with backend APIs.

Main modules:

- components – reusable UI components
- views – pages for each user role

- stores – Pinia state management
 - services – API communication
-

Implemented Features

Authentication

- JWT-based authentication
- Role-based access control

Admin Features

- Approve or reject companies
- Approve placement drives
- View system analytics dashboard

Company Features

- Create placement drives
- View applicants
- Update candidate status

Student Features

- Browse eligible drives
- Apply for placement drives
- Track application status

Analytics

- Dashboard charts using Chart.js for system statistics.

Caching

- Redis caching for frequently accessed data like drive listings and dashboard statistics.

Background Tasks

- Automated reminders for upcoming deadlines
 - Export application data as CSV
 - Monthly placement statistics reports
-

8. Video Presentation

Drive Link:

<https://drive.google.com/file/d/1fdltVSXFauxkT4ChnxFIUWcAJLPZy2-e/view?usp=sharing>

The video demonstrates the system workflow including login, drive creation, application submission, and dashboard analytics.